ECE420 Quartus Tutorial03: 2-bit Adder Design GLaw

VHDL Design Entry Method & Simulation with Testbench

Notation:

To simplify what one should click to perform a task, the following form of notation will be used:

Menu item → Menu item → … → Menu item

Example:

File → Project Wizard means "on the File menu, click New Project Wizard"

Objective

To learn to use Quartus to develop and simulate a structured 2-bit adder using VHDL Design Entry Method

Development Tools

Quartus 11

Hardware

DE2 board, USB Blaster, and USB cable.

References

Altera Quartus Handbooks Volume 1, 2, and 3 available at www.altera.com or at the course website www.csun.edu/~glaw

Launch Quartus II Software

1. Launch the Quartus II software.
2. File → Project Wizard (on the File menu, click New Project Wizard)
3. New Project Wizard: Introduction windows appears. It shows you the five steps you may need to go through in setting up a new project. You have nothing to do in this window, hence click **Next**.
4. New Project Wizard: Directory, Name, Top-Level Entity window appears.

   a. Enter the working project directory in your Z drive, for example, z:\...\Tutorial03

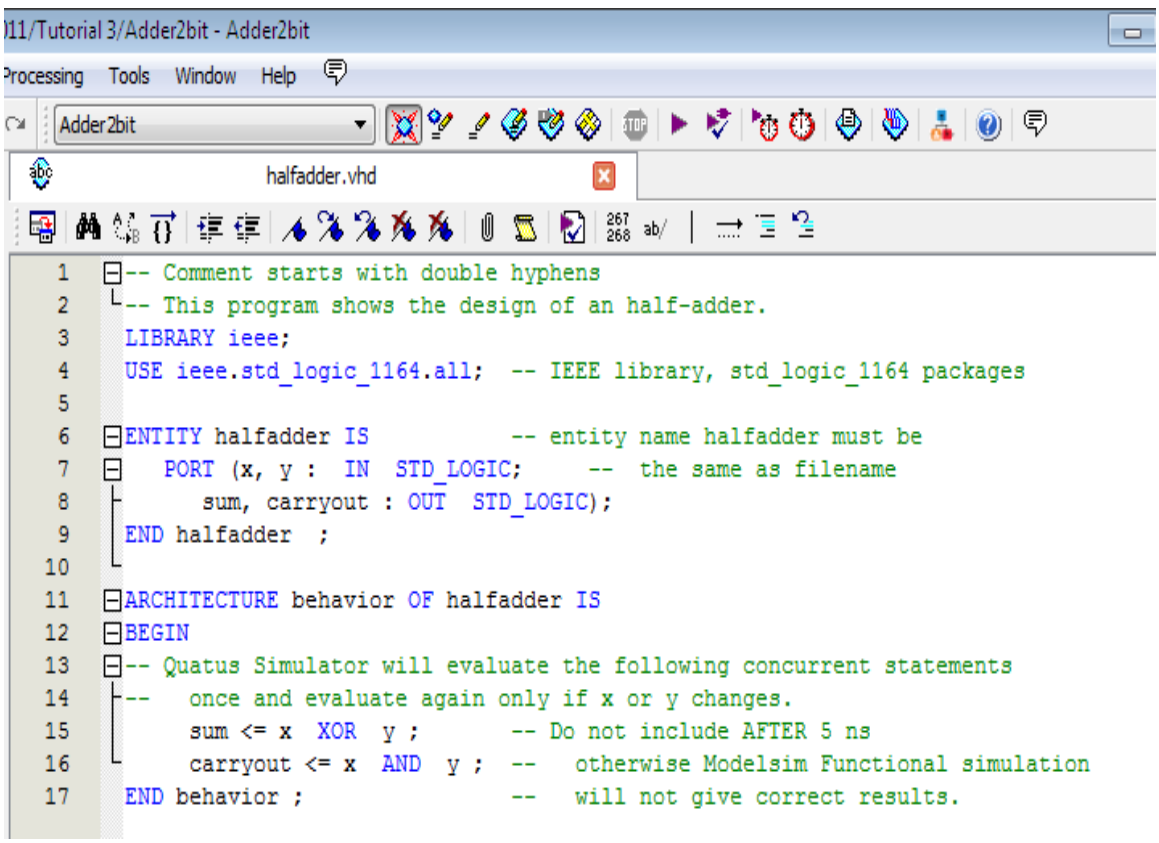
   b. Enter project name: adder2bit. As you type adder2bit name, Top-Level Entity name will be automatically filled with adder2bit.

   c. Click next. When asked to create z:\...\Tutorial3 directory, click **Ye**s.
5. New Project Wizard: Add Files window appears. You have no files to add now, but need to add files later. Hence, just click **Next**.

6. New Project Wizard: Family & Device Settings window appears.
   Scroll up/down the family choices and choose Cyclone II family. Scroll up/down the available devices and choose EP2C35F672C6. Leave other options as they are. Click **Finish**.

Create a Half Adder

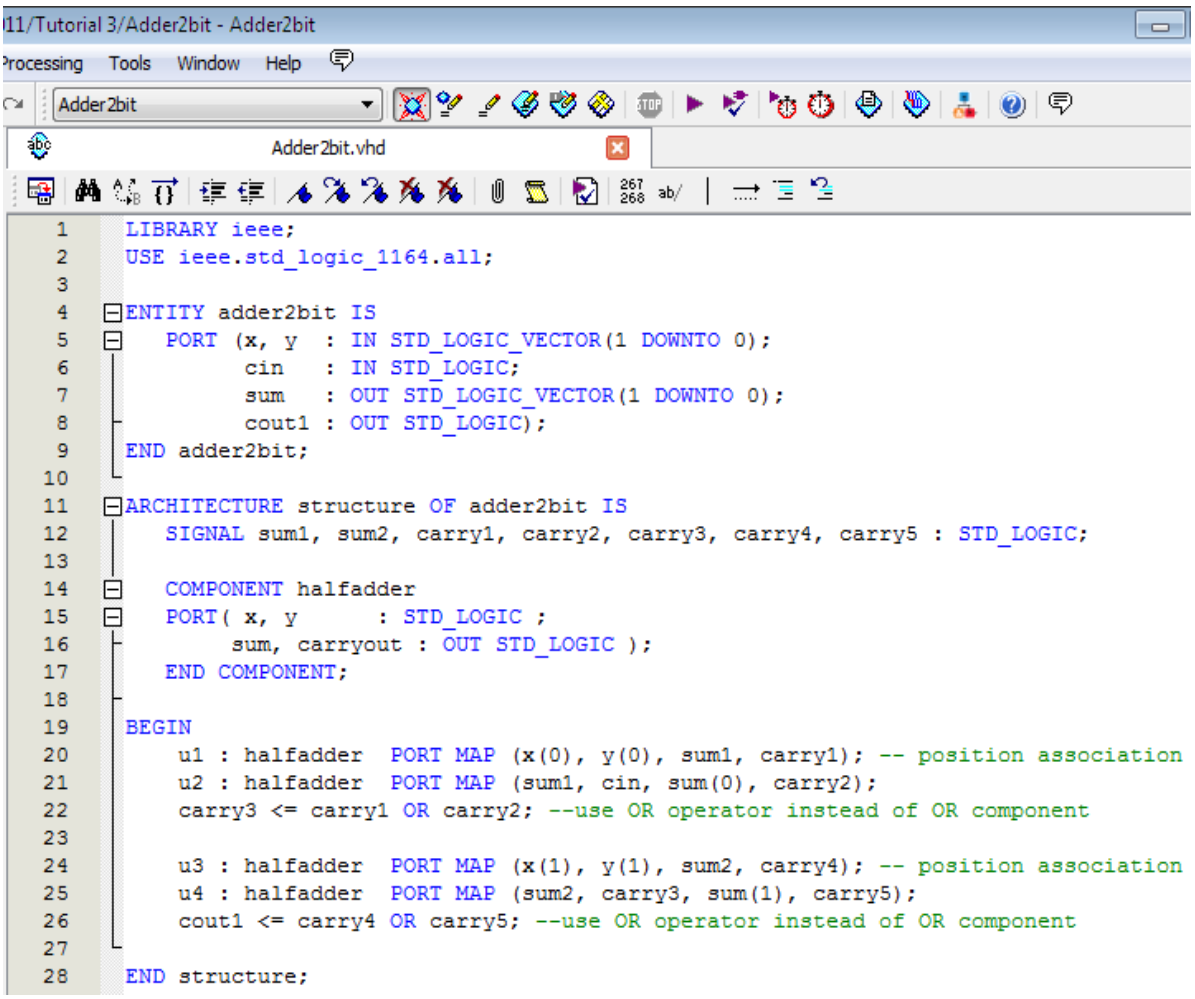
1. File → New. New file option window appears.
2. Select VHDL File. Click OK. Quartus Text Editor window appears.
3. Starting from line 1 in the Quartus Text Editor window, type the Halfadder VHDL code example 1 in the lecture note. To save time, you could copy the contents of halfadder.vhd available on the course website and paste it in the editor window.
4. Now, your VHDL file must look similar to the one shown below:



5. Save the VHDL file as halfadder.vhd.
6. Add halfadder.vhd to the project. Project → Add Current File to Project.

Create a 2-bit adder using two halfadders and one OR gates.

1. File → New. New file option window appears.
2. Select VHDL File. Click OK. Quartus Text Editor window appears.
3. Starting from line 1 in the Quartus Text Editor window, type the VHDL code shown below. To save time, you could copy the contents of adder2bit.vhd available on the course website and paste it in the editor window. Make sure that you understand how adder2bit.vhd uses halfadders to implement 2-bit adder. You could refer to adder2bit.bdf in Tutorial02 to see the graphical connections of the halfadders.
4. Now, your VHDL file must look similar to the one shown below:

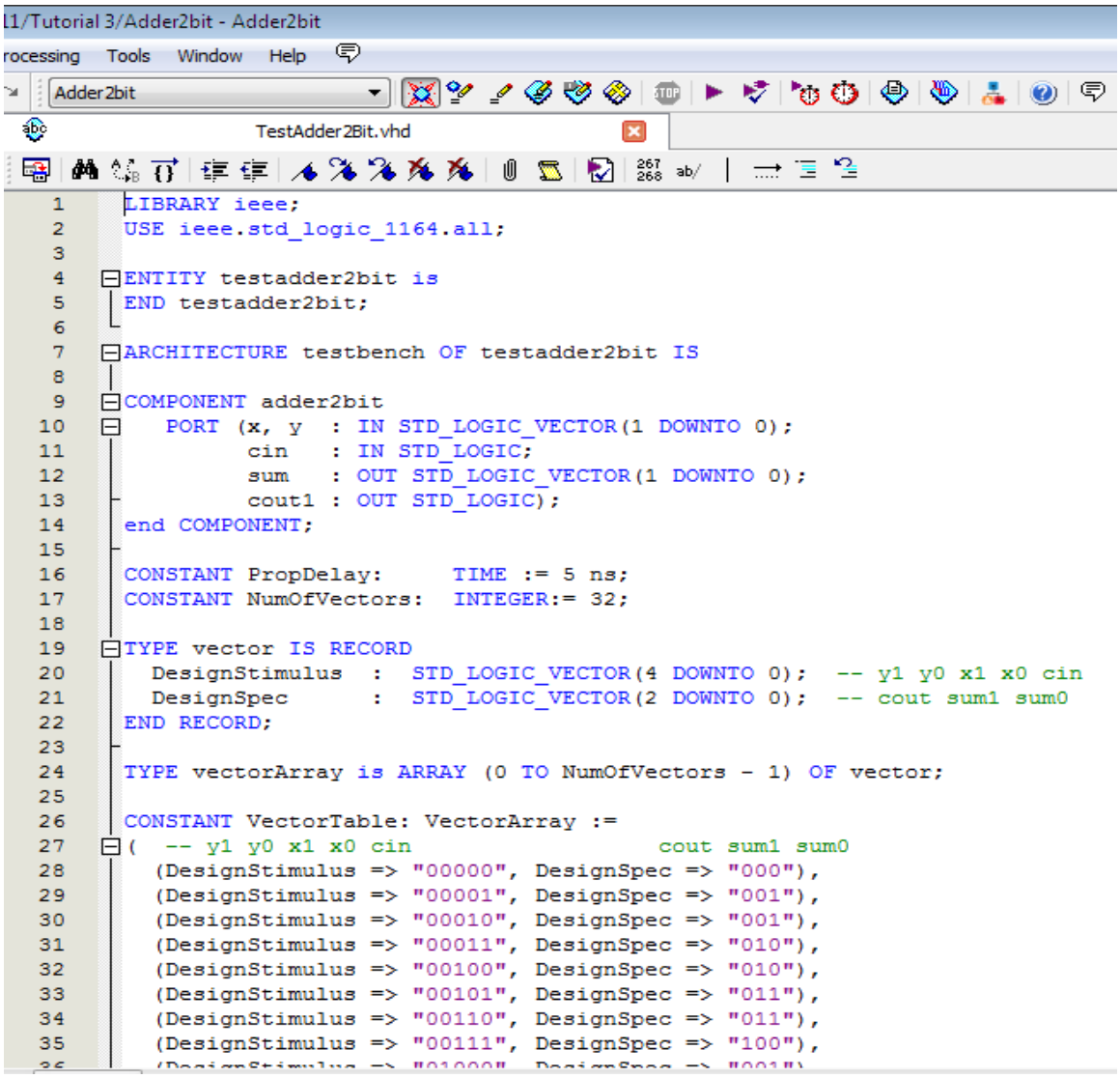



```vhdl
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.all;
3
4  ENTITY adder2bit IS
5     PORT (x, y  : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
6           cin   : IN STD_LOGIC;
7           sum   : OUT STD_LOGIC_VECTOR(1 DOWNTO 0);
8           cout1 : OUT STD_LOGIC);
9  END adder2bit;
10
11 ARCHITECTURE structure OF adder2bit IS
12    SIGNAL sum1, sum2, carry1, carry2, carry3, carry4, carry5 : STD_LOGIC;
13
14    COMPONENT halfadder
15    PORT( x, y       : STD_LOGIC ;
16          sum, carryout : OUT STD_LOGIC );
17    END COMPONENT;
18
19 BEGIN
20     u1 : halfadder  PORT MAP (x(0), y(0), sum1, carry1); -- position association
21     u2 : halfadder  PORT MAP (sum1, cin, sum(0), carry2);
22     carry3 <= carry1 OR carry2; --use OR operator instead of OR component
23
24     u3 : halfadder  PORT MAP (x(1), y(1), sum2, carry4); -- position association
25     u4 : halfadder  PORT MAP (sum2, carry3, sum(1), carry5);
26     cout1 <= carry4 OR carry5; --use OR operator instead of OR component
27
28 END structure;
```

5. Save your 2-bit adder file and name it adder2bit.vhd. Note that the top design name must be the same as the project name. Here, the top design is a 2-bit adder and our project name is adder2bit.

Create the Testbench

1. File → New. New file option window appears.
2. Select VHDL File. Click OK. Quartus Text Editor window appears.
3. The testadder2bit.vhd is a larger VHDL program and you may not want to type the codes line by line. Instead, you may want to copy the contents of testadder2bit.vhd available on the course website and paste it in the editor window.
4. Now, your VHDL file must look similar to the one shown below:

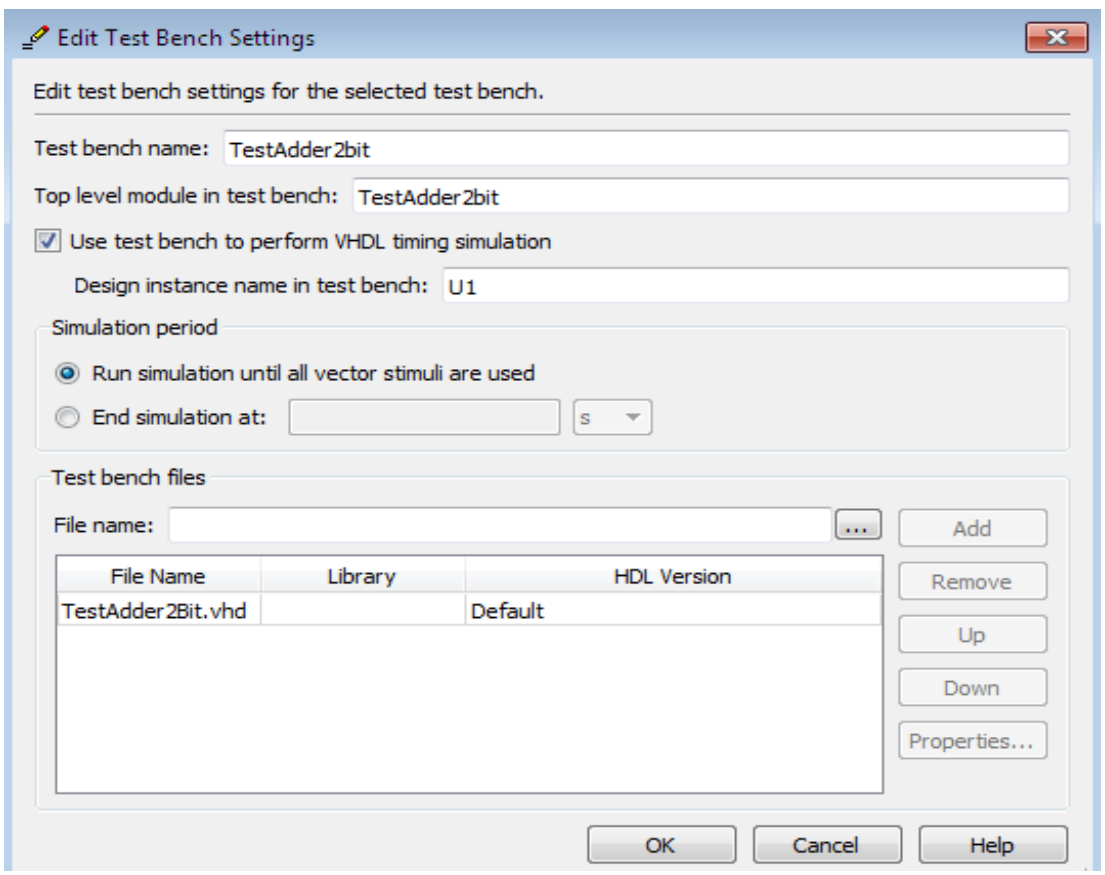



```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY testadder2bit is
END testadder2bit;

ARCHITECTURE testbench OF testadder2bit IS

COMPONENT adder2bit
   PORT (x, y  : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
         cin   : IN STD_LOGIC;
         sum   : OUT STD_LOGIC_VECTOR(1 DOWNTO 0);
         cout1 : OUT STD_LOGIC);
end COMPONENT;

CONSTANT PropDelay:     TIME := 5 ns;
CONSTANT NumOfVectors:  INTEGER:= 32;

TYPE vector IS RECORD
  DesignStimulus  :  STD_LOGIC_VECTOR(4 DOWNTO 0);  -- y1 y0 x1 x0 cin
  DesignSpec      :  STD_LOGIC_VECTOR(2 DOWNTO 0);  -- cout sum1 sum0
END RECORD;

TYPE vectorArray is ARRAY (0 TO NumOfVectors - 1) OF vector;

CONSTANT VectorTable: VectorArray :=
(  -- y1 y0 x1 x0 cin                    cout sum1 sum0
  (DesignStimulus => "00000", DesignSpec => "000"),
  (DesignStimulus => "00001", DesignSpec => "001"),
  (DesignStimulus => "00010", DesignSpec => "001"),
  (DesignStimulus => "00011", DesignSpec => "010"),
  (DesignStimulus => "00100", DesignSpec => "010"),
  (DesignStimulus => "00101", DesignSpec => "011"),
  (DesignStimulus => "00110", DesignSpec => "011"),
  (DesignStimulus => "00111", DesignSpec => "100"),
```

5. Save your testbench file and name it testadder2bit.vhd.

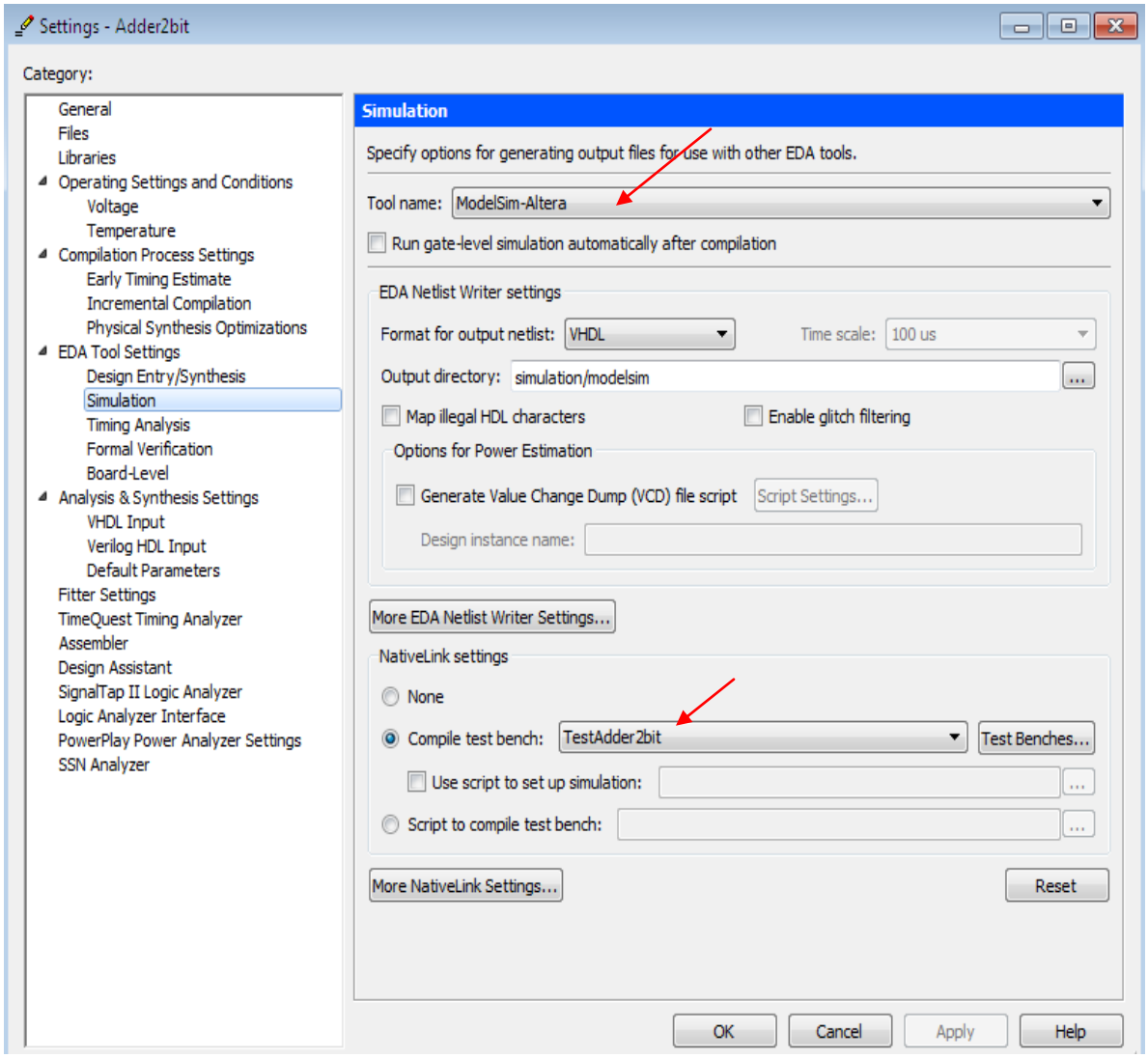
Set Quartus to Use ModelSim Simulator

1. Assignments → Settings. Adder2bit settings window appears.
2. Under EDA Tool setting, click simulation. Browse Tool Name and choose Modelsim-Altera.
3. Mark (Select) Compile Test bench and click Test Benches tab. Edit Test Bench Settings windows appears.
4. In the Test bench name box and in the Top level module in test bench box, type testadder2bit.
5. Check " Use test bench to perform VHDL timing simulation" box and type U1 in Design instance name in test bench.
6. Mark (select) Run simulation until all vector stimuli are used
7. In the file name box, browse to the folder that contains testadder2bit.vhd and click Add.
8. Your test bench setting should look similar to the one shown:



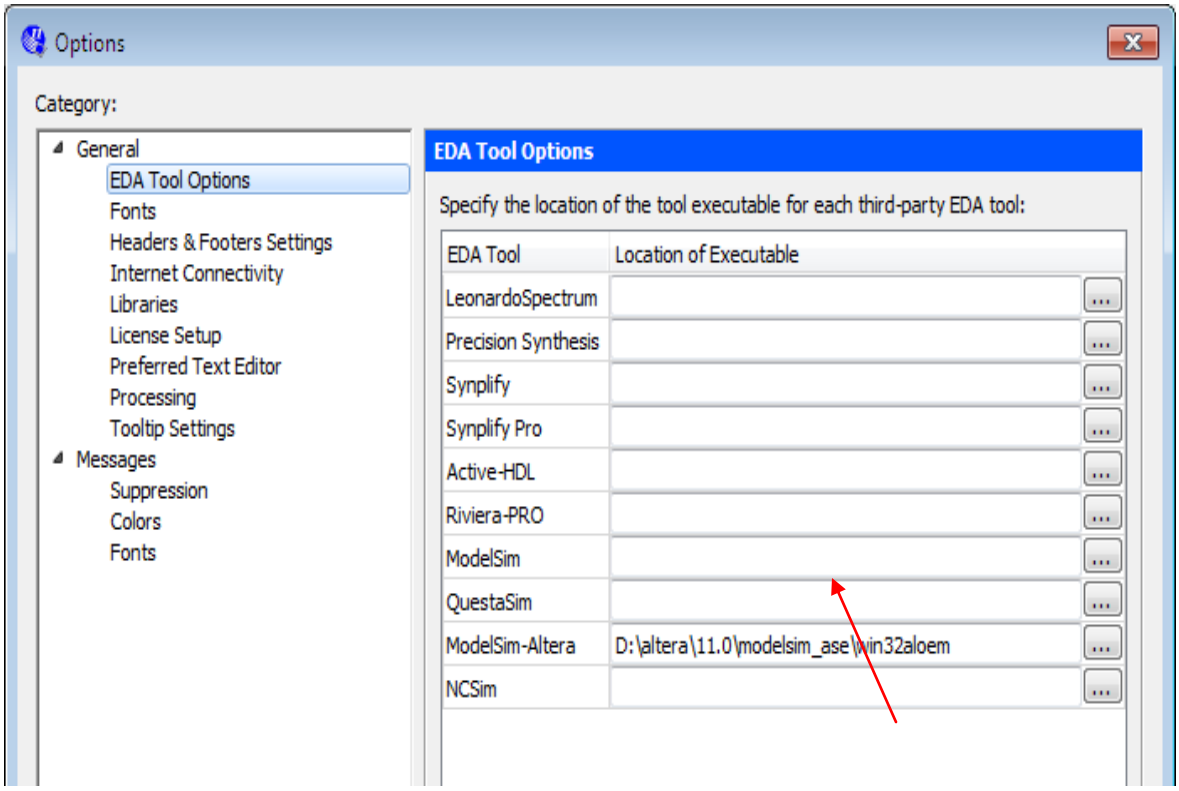
9. Click OK.

10. Your Settings windows should similar to the one shown:



11. Click OK.

12. Tools → Options. Tools Option window appears. Under Category /General, select EDA Tool Options. In the Modelsim-Altera field, browse the folders and locate win32aloem folder for the Modelsim executable file. Click OK.



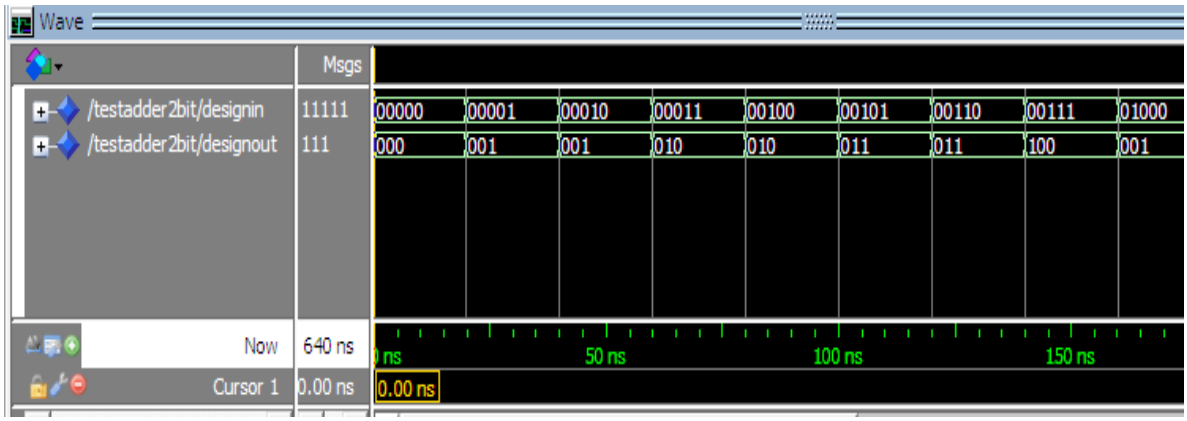
Compile the 2-bit adder

Processing → Start Compilation. If there is errors in the message window, fix the errors and compile the design again. Ignore the warnings.

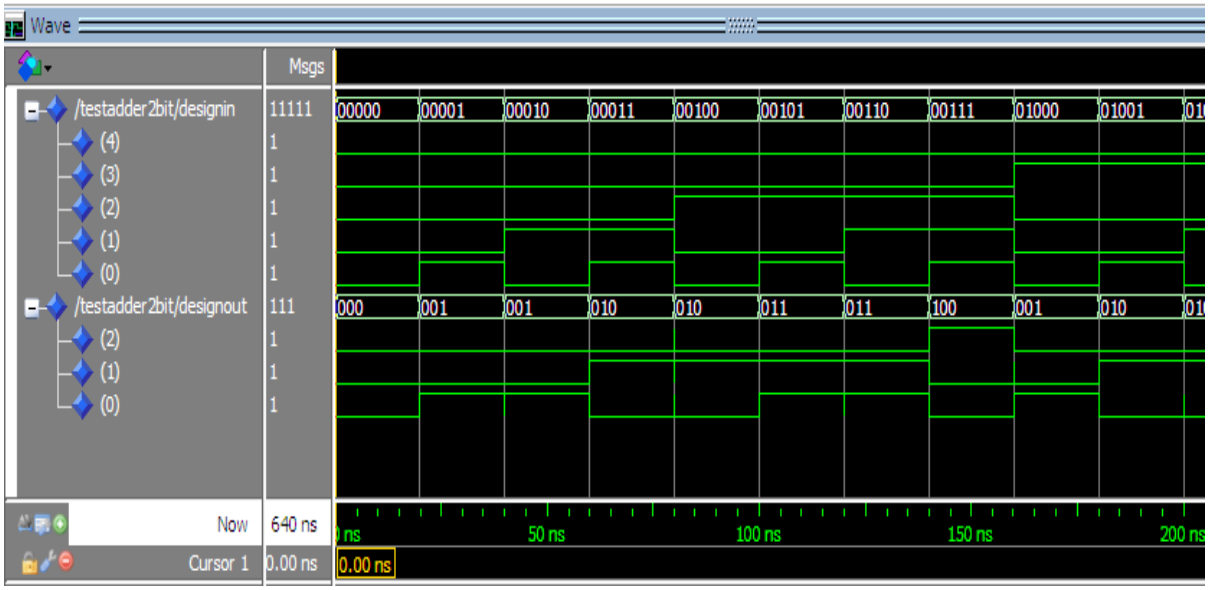
Simulate the 2-bit adder (Functional Simulation)

1. Tools → Run EDA Simulation Tool → EDA RTL Simulation
2. Wait a few minutes for the Modelsim windows to open. Click the Modelsim icon on the task bar, if the Modelsim window does not open automatically.

3. On the wave window, Zoom in as much as necessary to see the simulation output clearly. To save time, you could Zoom Full first and then Zoom-in a few times. Slide the time bar to 0 ns and manually verify that the additions are correct. You can also scroll the Transcript window up and down, and see if any errors are reported.



4. You could see the binary values of the adder inputs and outputs by expanding the inputs and outputs. That is, click ⊞.



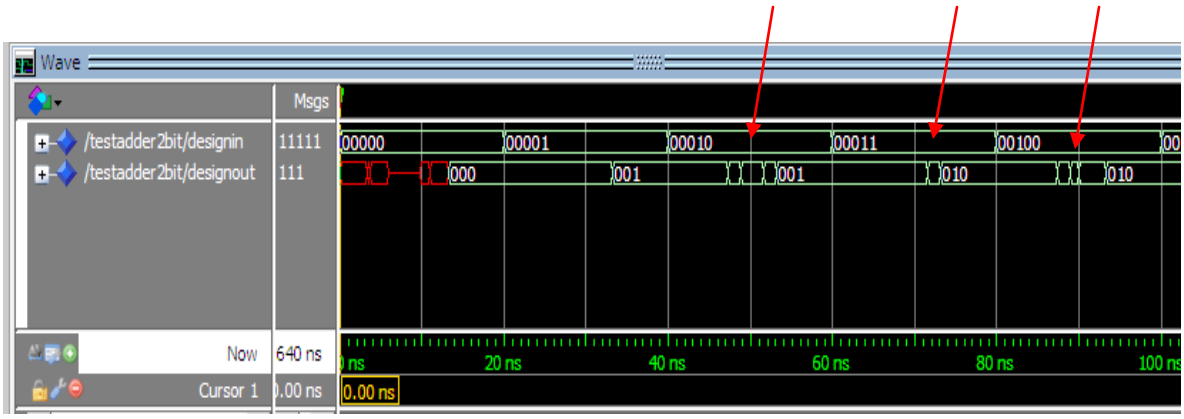
Note that because you just did functional simulation, the input and output changes at the same time with no delay.

5. Save the simulation output to Testadder2bitFunctional.do.

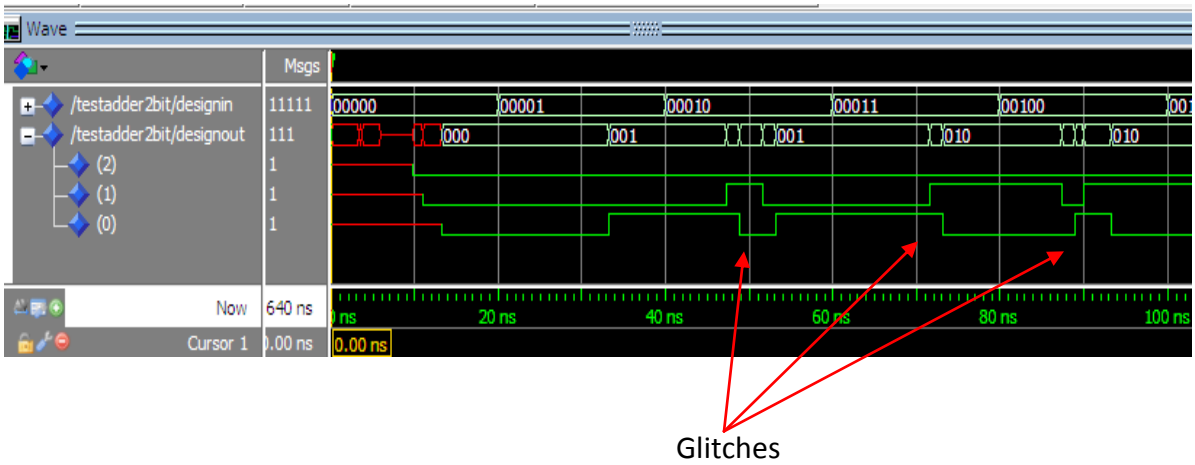
Simulate the 2-bit adder (Timing Simulation)

6. Close Modelsim. If you forget to close Modelsim, you will get errors in the later steps.
7. Tools → Run EDA Simulation Tool → EDA Gate Level Simulation
8. On the gate Level Simulation window, you may choose either slow model or fast model and then click OK.
9. Wait a few minutes for the Modelsim windows to open. Click the Modelsim icon on the task bar, if the Modelsim window does not open automatically.

10. On the wave window, Zoom in as much as necessary to see the simulation output clearly. To save time, you could Zoom Full first and then Zoom-in a few times. Slide the time bar to 0 ns and manually verify that the additions are correct. You can also scroll the Transcript window up and down, and see if any errors are reported.



Note the undefined outputs between 0 to 13 ns and the glitches around 50ns, 74ns, and 88 ns. If you expand the outputs, you may be able to see the glitches more clearly in binary.



11. Save the simulation output to Testadder2bitTiming.do.